Laboratorio 3: Sensores y Librerias

- Materia: Sistemas de instrumentalización - Sistemas Embebidos - Famaf
- Alumno: José Lorenzo Canovas
- Año: 2026
- Microcontrolador utilizado: STM32F407G-Disc1

> Nota: Las explicaciones de las implementaciones se colocaron en comentarios. Para optimizar la lectura del informe y evitar redundancias, las explicaciones de código y conclusiones de los primeros ejercicios se aplican a los desafíos que les siguen, detallando en estos últimos únicamente las modificaciones y comportamientos novedosos.

3.1 - Desafío 1: HC-SR04

Añadir y utilizar esta librería fue relativamente simple, ya que lo vimos en clases. Tuve que aprender como añadir los paths para que el STM32CubeIDE me tomase los includes. En un comienzo no me funcionaba, ya que no se tomaba unos ms entre medición, lo cual hacia que los leds tuviesen un comportamiento no deseado, por lo que la medición actualmente se realiza cada 100ms. Ademas, termine colocando un timeout para que se haga de nuevo la medición en caso de algún problema.

```
// ...
#include "HC_SR04.h"

//...
HCSR04_HandleTypeDef sensor;

// ...
int main(void)
{
    // ...
    // Configuración necesaria para el sensor
    HAL_TIM_IC_Start_IT(&htim2, TIM_CHANNEL_1);
    sensor.htim = &htim2;
    sensor.IC_Channel = TIM_CHANNEL_1;
    sensor.Trig_Pin = Trigger_Pin;
    sensor.Trig_Port = Trigger_GPIO_Port;
    hcsr04_init(&sensor);

    // Variables para indicar si se esta realizando una medición
    // y cuando fue la ultima medición
    uint8_t is_measurement = 0;
    uint32_t last_tick = 0;
    while (1)
    {
        if(!is_measurement && (HAL_GetTick() - last_tick > 100)) {
            // Se comienza una nueva medición
            is_measurement = 1;
            hcsr04_start_measurement(&sensor);
```

```
        } else if (hcsr04_is_data_ready(&sensor)) {
            // Los datos ya estan, por lo que termina la medición y se
realiza
            // la lógica de los leds
            is_measurement = 0;
            last_tick = HAL_GetTick();

            float distance = hcsr04_get_distance_cm(&sensor);

            HAL_GPIO_WritePin(LD5_GPIO_Port,
LD3_Pin|LD4_Pin|LD5_Pin|LD6_Pin, GPIO_PIN_RESET);
            if (distance < 20.0) {
                // Enciende el LED Rojo (Zona de Peligro).
                HAL_GPIO_WritePin(LD5_GPIO_Port, LD5_Pin, GPIO_PIN_SET);
            } else if (distance < 50.0) {
                // Enciende el LED Naranja (Zona de Precaución).
                HAL_GPIO_WritePin(LD3_GPIO_Port, LD3_Pin, GPIO_PIN_SET);
            } else {
                // Enciende el LED Verde (Zona Segura).
                HAL_GPIO_WritePin(LD4_GPIO_Port, LD4_Pin, GPIO_PIN_SET);
            }
        } else if (is_measurement && (HAL_GetTick() - last_tick > 150)) {
                // TIMEOUT DE SEGURIDAD: Pasaron más de 150ms esperando el
eco,
                // por lo que se resetean la flag de medición
                is_measurement = 0;
                last_tick = HAL_GetTick();

                // Limpiar channel del timer
                HAL_TIM_IC_Stop_IT(&htim2, TIM_CHANNEL_1);
            }
    }
}
// ...

void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim){
    hcsr04_tim_ic_callback(&sensor, htim);
}
// ...
```

3.2 - Desafío 2: LM35

A diferencia del desafío anterior, no coloque un timeout a ningún sensor ya que este código estaba pensado principalmente para revisarlo en modo debug. En una aplicación real, seria recomendable hacerlo. El código a continuación solo muestra el desafió no bloqueante, ya que el código necesario para ejecutarlo en modo bloqueante es trivial.

```
// ...
#include "lm35.h"

//...
```

```
LM35_HandleTypeDef lm35_sensor_1;
LM35_HandleTypeDef lm35_sensor_2;

// ...
int main(void)
{
    // ...
    // Configuración necesaria para el sensor
    LM35_Init(&lm35_sensor_1, &hadc1, ADC_CHANNEL_1);
    LM35_Init(&lm35_sensor_2, &hadc1, ADC_CHANNEL_2);

    // Variables para indicar si se esta realizando una medición
    // y cuando fue la ultima medición
    uint8_t lm35_1_is_measurement = 0;
    uint32_t lm35_1_last_tick_read = 0;
    float lm35_1_temp = 0.0f;

    uint8_t lm35_2_is_measurement = 0;
    uint32_t lm35_2_last_tick_read = 0;
    float lm35_2_temp = 0.0f;
    while (1)
    {
        if (!lm35_1_is_measurement && (HAL_GetTick() -
lm35_1_last_tick_read > 100)) {
            // Se comienza una nueva medición
            lm35_1_is_measurement = 1;
            LM35_Start_Conversion(&lm35_sensor_1);
        } else if (LM35_Get_Temperature(&lm35_sensor_1, &lm35_1_temp)) {
            // Los datos ya estan, por lo que termina la medición
            lm35_1_is_measurement = 0;
            lm35_1_last_tick_read = HAL_GetTick();

            lm35_1_temp += 0.00001f;
        }

        if (!lm35_2_is_measurement && (HAL_GetTick() -
lm35_2_last_tick_read > 100)) {
            // Se comienza una nueva medición
            lm35_2_is_measurement = 1;
            LM35_Start_Conversion(&lm35_sensor_2);
        } else if (LM35_Get_Temperature(&lm35_sensor_2, &lm35_2_temp)) {
            // Los datos ya estan, por lo que termina la medición
            lm35_2_is_measurement = 0;
            lm35_2_last_tick_read = HAL_GetTick();

            lm35_2_temp += 0.00001f;
        }
    }
}
// ...
```

3.3 - Desafío 3: El Acelerómetro

Para mi, el principal desafió que tuve fue buscar el código para hacerlo funcionar (un LLM ayudo) y luego fue ver donde meter los archivos. Aunque luego en clases, gracias a la explicación del profe, me di cuenta que hice cualquiera.

Sobre el código, tuve que ponerme a probar y jugar con el micro para ver que umbral ponerle a los ejes x e y. En el acelerómetro, termine por no utilizar el led azul, ya que se ponía a titilar por el uso del heartbeat. E intentar poner lógica para que el acelerómetro no me utilizarse el led azul cuando el heartbeat esta apagado, era lo mismo que no utilizarlo.

Cuando se tenia el heartbeat bloqueante, entonces el acelerómetro no reaccionaba a los cambios de la inclinación durante esos 500ms. Y esto seria lo mismo que hacer mediciones cada 500ms. Por lo cual, si en una aplicación sucede algo similar, las consecuencias serian:

1. La aplicación no funciona como se espera, ya que se pierde tiempo de muestreo.
2. Perdida de dinero si se busco hardware rápido par hacer muchas mediciones, ya que se termina realizando pocas por bloqueos del software (posiblemente comprable a un hardware mas simple y económico).

```
// ...
#include "stm32f4_discovery_accelerometer.h"

//...
int16_t accel_data_xyz[3];

// ...
int main(void)
{
    // ...
    // Configuración necesaria para el sensor
    BSP_ACCELERO_Init();

    // Variables para indicar cuando fue la ultima medición
    uint32_t accel_last_tick_read = 0;
    const int16_t ACCEL_X_UMBRAL = 100;
    const int16_t ACCEL_Y_UMBRAL = 100;

    // Variables para el heartbeat
    uint8_t heartbeat_is_blinking = 0;
    uint32_t heartbeat_last_tick_read = 0;

    while (1)
    {
        if (HAL_GetTick() - accel_last_tick_read > 50) {
            BSP_ACCELERO_GetXYZ(accel_data_xyz);

            int16_t x = accel_data_xyz[0];
            int16_t y = accel_data_xyz[1];
            // int16_t z = accel_data[2];

            if (x > ACCEL_X_UMBRAL) {
                HAL_GPIO_WritePin(LD4_GPIO_Port, LD4_Pin, GPIO_PIN_SET);
```

```
                HAL_GPIO_WritePin(LD5_GPIO_Port, LD5_Pin, GPIO_PIN_RESET);
            } else if (x < -ACCEL_X_UMBRAL) {
                HAL_GPIO_WritePin(LD5_GPIO_Port, LD5_Pin, GPIO_PIN_SET);
                HAL_GPIO_WritePin(LD4_GPIO_Port, LD4_Pin, GPIO_PIN_RESET);
            } else if (y > ACCEL_Y_UMBRAL) {
                // No se enciende el led azul, ya que es utilizado por el
heartbeat
                HAL_GPIO_WritePin(LD3_GPIO_Port, LD3_Pin, GPIO_PIN_RESET);
            } else if (y < -ACCEL_Y_UMBRAL) {
                HAL_GPIO_WritePin(LD3_GPIO_Port, LD3_Pin, GPIO_PIN_SET);
                HAL_GPIO_WritePin(LD6_GPIO_Port, LD6_Pin, GPIO_PIN_RESET);
            } else {
                HAL_GPIO_WritePin(GPIOD, LD3_Pin|LD4_Pin|LD5_Pin|LD6_Pin,
GPIO_PIN_RESET);
            }
        }

        if (HAL_GetTick() - heartbeat_last_tick_read > 500) {
            // Si pasaron 500ms se cambia el estado del heartbeat
            heartbeat_last_tick_read = HAL_GetTick();
            heartbeat_is_blinking = !heartbeat_is_blinking;
        }

        if (heartbeat_is_blinking) {
            HAL_GPIO_WritePin(LD6_GPIO_Port, LD6_Pin, GPIO_PIN_SET);
        } else {
            HAL_GPIO_WritePin(LD6_GPIO_Port, LD6_Pin, GPIO_PIN_RESET);
        }
    }
}
// ...
```

3.4 - Desafío 4: El Super-loop y la Integración Total

Termine moviendo numero mágicos a macros (intervalos de muestreo, umbrales de los ejes del acelerómetro) para mejorar legibilidad. Coloque lógica para que si se esta parpadeando el led rojo (por obstáculo a una distancia menor de 15cm), entonces el led rojo no se utilize en el acelerómetro (evitar parpadeos no deseados).

```
// ...
#include "stm32f4_discovery_accelerometer.h"
#include "HC_SR04.h"
#include "lm35.h"

// ...
#define ACCEL_MESURAMENT_INTERVALE_MS 50
#define ACCEL_X_UMBRAL 100
#define ACCEL_Y_UMBRAL 100

#define HCSR04_MESURAMENT_INTERVALE_MS 100
#define HCSR04_TIMEOUT_ECO_MS 150
```

```c
#define LM35_MESURAMENT_INTERVALE_MS 1000
#define ALARM_INTERVALE_MS 100

// ...
HCSR04_HandleTypeDef hcsr04_sensor;
LM35_HandleTypeDef lm35_sensor_1;
int16_t accel_data_xyz[3];

// ...
int main(void)
{
    // ...
    // Configuración necesaria para los sensores
    BSP_ACCELERO_Init();

    HAL_TIM_IC_Start_IT(&htim2, TIM_CHANNEL_1);
    hcsr04_sensor.htim = &htim2;
    hcsr04_sensor.IC_Channel = TIM_CHANNEL_1;
    hcsr04_sensor.Trig_Pin = Trigger_Pin;
    hcsr04_sensor.Trig_Port = Trigger_GPIO_Port;
    hcsr04_init(&hcsr04_sensor);

    LM35_Init(&lm35_sensor_1, &hadc1, ADC_CHANNEL_1);

    // Variables utilizadas
    uint32_t accel_last_tick_read = 0;

    uint8_t hcsr04_is_measurement = 0;
    uint32_t hcsr04_last_tick_read = 0;

    uint8_t lm35_1_is_measurement = 0;
    uint32_t lm35_1_last_tick_read = 0;
    float lm35_1_temp = 0.0f;

    uint8_t is_active_obstacle_alarm = 0;
    uint8_t alarm_is_blinking = 0;
    uint32_t alarm_last_tick_read = 0;

    while (1) {
        // Sensor ultrasonico HC-SR04
        if (!hcsr04_is_measurement
                && (HAL_GetTick() - hcsr04_last_tick_read >
HCSR04_MESURAMENT_INTERVALE_MS)) {
            hcsr04_is_measurement = 1;

            hcsr04_start_measurement(&hcsr04_sensor);
        } else if (hcsr04_is_data_ready(&hcsr04_sensor)) {
            hcsr04_is_measurement = 0;
            hcsr04_last_tick_read = HAL_GetTick();

            float distance = hcsr04_get_distance_cm(&hcsr04_sensor);

            if (distance > 15.0) {
                // Enciende el LED Rojo (Zona de Peligro).
```

```
                is_active_obstacle_alarm = 0;
                alarm_is_blinking = 0;
            } else {
                is_active_obstacle_alarm = 1;
            }
        } else if (hcsr04_is_measurement
                && (HAL_GetTick() - hcsr04_last_tick_read >
HCSR04_TIMEOUT_ECO_MS)) {
            // TIMEOUT DE SEGURIDAD (más de 150ms esperando el eco)

            hcsr04_is_measurement = 0; // Fuerza a reintentar en el
siguiente ciclo
            hcsr04_last_tick_read = HAL_GetTick();

            // Limpiar channel del timer
            HAL_TIM_IC_Stop_IT(&htim2, TIM_CHANNEL_1);
        }

        // Sensor de temperatura LM35
        if (!lm35_1_is_measurement
                && (HAL_GetTick() - lm35_1_last_tick_read >
LM35_MESURAMENT_INTERVALE_MS)) {
            lm35_1_is_measurement = 1;

            LM35_Start_Conversion(&lm35_sensor_1);
        } else if (LM35_Get_Temperature(&lm35_sensor_1, &lm35_1_temp)) {
            lm35_1_is_measurement = 0;
            lm35_1_last_tick_read = HAL_GetTick();

            lm35_1_temp += 0.0001f;
        }

        // Acelerometro
        if (HAL_GetTick() - accel_last_tick_read >
ACCEL_MESURAMENT_INTERVALE_MS) {
            BSP_ACCELERO_GetXYZ(accel_data_xyz);

            int16_t x = accel_data_xyz[0];
            int16_t y = accel_data_xyz[1];
            // int16_t z = accel_data[2];

            if (x > ACCEL_X_UMBRAL) {
                HAL_GPIO_WritePin(LD4_GPIO_Port, LD4_Pin, GPIO_PIN_SET);
                HAL_GPIO_WritePin(LD5_GPIO_Port, LD5_Pin, GPIO_PIN_RESET);
            } else if (x < -ACCEL_X_UMBRAL) {
                HAL_GPIO_WritePin(LD5_GPIO_Port, LD5_Pin, GPIO_PIN_SET);
                HAL_GPIO_WritePin(LD4_GPIO_Port, LD4_Pin, GPIO_PIN_RESET);
            } else if (y > ACCEL_Y_UMBRAL) {
                if (!is_active_obstacle_alarm) {
                    HAL_GPIO_WritePin(LD6_GPIO_Port, LD6_Pin,
GPIO_PIN_SET);
                }
                HAL_GPIO_WritePin(LD3_GPIO_Port, LD3_Pin, GPIO_PIN_RESET);
            } else if (y < -ACCEL_Y_UMBRAL) {
```

```
                HAL_GPIO_WritePin(LD3_GPIO_Port, LD3_Pin, GPIO_PIN_SET);
                HAL_GPIO_WritePin(LD6_GPIO_Port, LD6_Pin, GPIO_PIN_RESET);
            } else {
                HAL_GPIO_WritePin(GPIOD, LD3_Pin|LD4_Pin|LD5_Pin|LD6_Pin,
GPIO_PIN_RESET);
            }
        }

        // Lógica del parpadeo del led rojo por obstáculo a menos de 15cm
        if (is_active_obstacle_alarm) {
            if (HAL_GetTick() - alarm_last_tick_read > ALARM_INTERVALE_MS)
{
                alarm_last_tick_read = HAL_GetTick();
                alarm_is_blinking = !alarm_is_blinking;
            }

            if (alarm_is_blinking) {
                HAL_GPIO_WritePin(LD5_GPIO_Port, LD5_Pin, GPIO_PIN_SET);
            } else {
                HAL_GPIO_WritePin(LD5_GPIO_Port, LD5_Pin, GPIO_PIN_RESET);
            }
        }
    }
}
// ...

void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim) {
    hcsr04_tim_ic_callback(&hcsr04_sensor, htim);
}
// ...
```